# CS 182 - HW 10 - Jie Wen

## 1. Kernelized Linear Attention (Part II)

The given parameters are:

- sequence length: $N$
- query/key dim: $D$
- value dim: $M$
- random feature dim: $D_{\text{random}}$ (denoted as $R$)

### (a) Approximating softmax attention with linearized kernel attention

The goal is to approximate

$$\text{sim}_{\text{softmax}}(q,k) = \exp\left(\frac{q^\top k}{\sigma^2}\right), \quad \text{with } \sigma^2 = \sqrt{D}.$$

#### (a.i) Rewrite softmax similarity using Gaussian kernel

The Gaussian kernel is

$$K_{\text{Gauss}}(q,k) = \exp\left(-\frac{|q-k|^2}{2\sigma^2}\right).$$

Using the identity

$$|q-k|^2 = |q|^2 + |k|^2 - 2q^\top k \quad \Rightarrow \quad q^\top k = \frac{|q|^2 + |k|^2 - |q-k|^2}{2},$$

the softmax similarity can be rewritten as:

$$\exp\left(\frac{q^\top k}{\sigma^2}\right) = \exp\left(\frac{|q|^2 + |k|^2 - |q-k|^2}{2\sigma^2}\right) = \exp\left(-\frac{|q-k|^2}{2\sigma^2}\right) \exp\left(\frac{|q|^2 + |k|^2}{2\sigma^2}\right).$$

So

$$\text{sim}_{\text{softmax}}(q,k) = e^{\frac{|q|^2}{2\sigma^2}} e^{\frac{|k|^2}{2\sigma^2}} K_{\text{Gauss}}(q,k)$$

with $K_{\text{Gauss}}(q,k) = \exp\left(-\frac{|q-k|^2}{2\sigma^2}\right)$. This is a product of the Gaussian kernel and two separate scalar factors.

#### (a.ii) Use random Fourier features to approximate softmax similarity and derive complexity

Given a random feature map $\phi_{\text{random}}$ of dimension $D_{\text{random}} = R$:

$$\phi_{\text{random}}(q) = \frac{1}{\sqrt{R}}\left[\sin(w_1^\top q), \ldots, \sin(w_R^\top q), \cos(w_1^\top q), \ldots, \cos(w_R^\top q)\right]^\top,$$

with $w_i \sim \mathcal{N}(0, \sigma^2 I_D)$. For the Gaussian kernel,

$$\mathbb{E}_{w_i}[\phi_{\text{random}}(q)^\top \phi_{\text{random}}(k)] = \exp\left(-\frac{|q-k|^2}{2\sigma^2}\right) = K_{\text{Gauss}}(q,k).$$

Using the result from part (i):

$$\text{sim}_{\text{softmax}}(q,k) = e^{\frac{|q|^2}{2\sigma^2}} e^{\frac{|k|^2}{2\sigma^2}} K_{\text{Gauss}}(q,k).$$

Approximate $K_{\text{Gauss}}$ via random features:

$$K_{\text{Gauss}}(q,k) \approx \phi_{\text{random}}(q)^\top \phi_{\text{random}}(k).$$

Define a **new feature map**:

$$\Phi(q) \coloneqq e^{\frac{|q|^2}{2\sigma^2}} \phi_{\text{random}}(q).$$

Then

$$\text{sim}_{\text{softmax}}(q,k) \approx \Phi(q)^\top \Phi(k).$$

Softmax attention becomes a linear (kernel) attention with feature map $\Phi$:

$$V_i' \approx \frac{\sum_{j=1}^N \Phi(Q_i)^\top \Phi(K_j) V_j}{\sum_{j=1}^N \Phi(Q_i)^\top \Phi(K_j)}.$$

This can be rewritten as:

- Let $R = D_{\text{random}}$.
- Define

$$A \coloneqq \sum_{j=1}^N \Phi(K_j)^\top V_j \in \mathbb{R}^{R \times M}, \quad b \coloneqq \sum_{j=1}^N \Phi(K_j)^\top \in \mathbb{R}^{R \times 1}.$$

  Then

$$V_i' \approx \frac{\Phi(Q_i) A}{\Phi(Q_i) b}$$

  where:
- $\Phi(Q_i)$ is $1 \times R$
- $A$ is $R \times M$
- $b$ is $R \times 1$.

### Computational cost for all $V'$

The cost to compute all outputs $V_1', \ldots, V_N'$ is determined. Let $R = D_{\text{random}}$.

1. **Compute $\Phi(K_j)$ and $\Phi(Q_i)$**
   For each vector (e.g., a key $K_j$):

   - Compute $w_r^\top K_j$ for all $r = 1, \ldots, R$: cost $\Theta(DR)$ per vector.
   - Apply sin and cos: $\Theta(R)$.
   - Multiply by scalar $e^{|K_j|^2/(2\sigma^2)}$: $\Theta(R)$.
   - The cost per $K_j$ is $\Theta(DR)$. Similarly per query.

   Total for all N queries and N keys:

$$\Theta(NDR) + \Theta(NDR) = \Theta(NDR).$$

2. **Compute $A = \sum_{j=1}^N \Phi(K_j)^\top V_j$**
   Each $\Phi(K_j)^\top V_j$ is an outer product:
   - $\Phi(K_j)^\top$ is $R \times 1$, $V_j$ is $1 \times M$, so the cost per $j$ is $\Theta(RM)$.
     Summing over $j = 1..N$:

$$\Theta(NRM).$$

3. **Compute $b = \sum_{j=1}^N \Phi(K_j)^\top$**
   Summing R-dimensional vectors:

$$\Theta(NR).$$

4. **Compute all outputs $V_i'$**
   For each $i$:
   - Numerator: $\Phi(Q_i)A$ is $(1 \times R)(R \times M)$: $\Theta(RM)$.
   - Denominator: $\Phi(Q_i)b$ is $(1 \times R)(R \times 1)$: $\Theta(R)$.
     Per token: $\Theta(RM)$. For all $N$ tokens:

$$\Theta(NRM).$$

### Overall complexity

The total cost is dominated by:

- Feature map computation: $\Theta(NDR)$
- Linearized attention core: $\Theta(NRM)$.
  So

$$\text{Cost} = \Theta\big(NR(D + M)\big)$$

  which is **linear in** $N$, versus softmax attention's $\Theta\big(N^2 \max(D, M)\big)$. When $R \ll N$, this is a substantial gain for long sequences.

## (b) Causal (autoregressive) masking with kernel attention

Causal masking implies that position $i$ can only attend to positions $j \leq i$. The standard attention formula becomes:

$$V_i' = \frac{\sum_{j=1}^i \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^i \text{sim}(Q_i, K_j)}$$

If $\text{sim}(Q_i, K_j) = \Phi(Q_i)^\top \Phi(K_j)$, then

$$V_i' = \frac{\sum_{j=1}^{i} \Phi(Q_i)^\top \Phi(K_j) V_j}{\sum_{j=1}^{i} \Phi(Q_i)^\top \Phi(K_j)}.$$

Factoring out $\Phi(Q_i)^\top$:

Define prefix sums:

$$S_i = \sum_{j=1}^{i} \Phi(K_j)^\top V_j, \quad Z_i = \sum_{j=1}^{i} \Phi(K_j)^\top.$$

- $S_i \in \mathbb{R}^{R \times M}$
- $Z_i \in \mathbb{R}^{R \times 1}$
  Then:

$$\sum_{j=1}^{i} \Phi(Q_i)^\top \Phi(K_j) V_j = \Phi(Q_i)^\top \Big( \sum_{j=1}^{i} \Phi(K_j)^\top V_j \Big) = \Phi(Q_i)^\top S_i,$$

$$\sum_{j=1}^{i} \Phi(Q_i)^\top \Phi(K_j) = \Phi(Q_i)^\top \Big( \sum_{j=1}^{i} \Phi(K_j)^\top \Big) = \Phi(Q_i)^\top Z_i.$$

The causal kernel attention formula becomes:

$$V_i' = \frac{\Phi(Q_i)^\top S_i}{\Phi(Q_i)^\top Z_i}$$

with $S_i, Z_i$ defined as in the problem.

### Complexity with causal masking

$S_i$ and $Z_i$ can be computed incrementally in a **single forward pass**:

- Initialize $S_0 = 0 \in \mathbb{R}^{R \times M}$, $Z_0 = 0 \in \mathbb{R}^{R \times 1}$.
- For $i = 1, \ldots, N$:
  - Update prefix sums:

    $$S_i = S_{i-1} + \Phi(K_i)^\top V_i$$

    outer product: cost $\Theta(RM)$ per i.

    $$Z_i = Z_{i-1} + \Phi(K_i)^\top$$

    vector add: cost $\Theta(R)$.
  - Compute output for position $i$:
    - Numerator: $\Phi(Q_i)^\top S_i$: $\Theta(RM)$.
    - Denominator: $\Phi(Q_i)^\top Z_i$: $\Theta(R)$.

The cost per position $i$ is $\Theta(RM)$. Over all positions:

$$\text{Causal kernel attention cost} = \Theta(NRM)$$

(plus $\Theta(NDR)$ for computing $\Phi(Q_i), \Phi(K_i)$ as before). Again, this is **linear in sequence length** $N$, even with causality, in contrast to quadratic softmax attention with a causal mask.

---

## 2. Coding Question: Hand-Design Transformers

### (a) Selecting by contents

### What both models are doing

- **Hand-designed Transformer**
  - $K_m = Q_m = sI$ (scaled identity, $s > 0$)
  - $V_m = I$.
  - For one-hot tokens A, B, C this means:
    - Keys and queries are **orthogonal one-hot vectors** (up to a common scale).
    - The attention scores ($QK^\top/\sqrt{d_k}$) are almost identity: each query has a large score with its own token and 0 with others.
    - The attention heatmap is therefore **almost a perfect identity matrix**, and `Out` ≈ input (identity map).
- **Learned Transformer**
  - After training, its $K_m/Q_m/V_m$ are **different numerically** but implement **the same high-level behavior**:
    - Keys/queries are not exact one-hots, but different tokens still map to **almost orthogonal directions**.
    - The attention heatmap is also **nearly identity** (each position attends mainly to itself; some off-diagonal mass remains).
    - `Out` rows are very close to the corresponding one-hot inputs but slightly “soft” (e.g. [0.91, 0.03, 0.06] instead of exactly).

These similarities make sense: to copy the sequence, the easiest strategy is “each token attends to itself”.

### Key differences and why they happen

1. **Structure of $K_m/Q_m$**
   - **Hand-designed:** exactly diagonal (scaled identity). Each content dimension is completely separated; pure orthogonality.
   - **Learned:** dense matrices with different scales in each row/column; some mixing between token dimensions.
   - **Reason:** attention is invariant to a change of basis in key/query space. If both $K_m$ and $Q_m$ are multiplied by the same invertible matrix $R$, the product $QK^\top$ can stay effectively the same (up to scaling), so training is free to pick any basis that minimizes loss; it is not forced to discover the literal identity matrix.
2. **Magnitude / scaling**
   - **Hand-designed:** a single global scale $s$ was used so logits for the correct token are very large relative to others, producing almost one-hot attention.
   - **Learned:** different rows/columns have different norms; attention logits are “large enough” but not uniform across dimensions.
   - **Reason:** gradient descent adjusts scales to balance between **sharpness of softmax** and **numerical stability**; many different scalings give almost zero MSE, so it just lands at some convenient point.
3. **Values $V_m$**
   - **Hand-designed:** exactly identity; V = input, so the attention-weighted average is literally an average of the original one-hots.
   - **Learned:** $V_m$ is dense and not exactly identity; each token’s value is some linear embedding.
   - **Reason:** as long as the output embedding is linearly separable and consistent, the model can learn a representation slightly different from raw one-hots but still close enough to minimize MSE. There is no constraint that $V_m$ must be identity.
4. **Outputs**
   - **Hand-designed Out:** almost perfectly one-hot (tiny deviations from softmax smoothing).
   - **Learned Out:** slightly more “blurred” toward other tokens, but still dominated by the correct dimension.
   - **Reason:** the objective is **MSE with noise and finite training steps**, not exact equality. A small blur that reduces gradient magnitudes can be preferred over a very sharp solution.

## (b) Selecting by position (copy the first token everywhere)

### Shared behavior

- **Task:** For any sequence, every position should output the **first token**.
- Both models ultimately implement:
  - **Attention weights:** each query attends almost entirely to **position 0**.
  - **Values:** the value vectors contain only the **token embedding** (positional components are ignored in V), so averaging over time just reproduces the content at position 0.
  - In the plots:
    - Hand-designed `Attn` is basically a column of ones at key index 0.
    - Learned `Attn` is very peaked on key 0 for all queries, with small spillover to other positions.
    - `Out` for both is [A, A, A] when the first token is A, etc., up to small numerical noise.

### Differences in parameters and intermediates

1. **Use of positional encodings in $K_m/Q_m$**
   - **Hand-designed:**
     - Positional encodings are **one-hot**; $K_m$ is nonzero only in the slot corresponding to “position 0”.
     - $Q_m$ is all ones, so Q ignores position; only K depends on position.
     - Result: attention scores differ only because keys differ by position; position 0 always gets the largest score regardless of content.
   - **Learned:**
     - Pos embeddings are not one-hot; their effective “basis” is rotated and scaled.
     - $K_m$ and $Q_m$ both mix **content and position**; some rows depend on token identity, others on position.
     - Yet the combination produces highest similarity between any query and the key at pos 0.
   - **Why:** training is free to entangle content and position, as long as the resulting **similarity matrix** has the right structure (column 0 highest). The hand design explicitly separates position and content for clarity; the learner finds a more entangled but functionally equivalent representation.
2. **Values $V_m$**
   - **Hand-designed:**
     - $V_m$ copies only the **first 3 dimensions** (token A/B/C) and discards all positional features; V = token one-hot.
   - **Learned:**
     - $V_m$ has non-zero entries in both token and positional dimensions.
     - Some positional information leaks into value vectors, but since attention is dominated by the first position for all queries, the model effectively outputs “position-0’s representation”, which correlates almost perfectly with its token.
   - **Why:** there’s no penalty for embedding some positional signal into V as long as, after training, the **mapping from those embeddings to targets** is correct. MSE allows many such equivalent embeddings.
3. **Attention sharpness**
   - **Hand-designed:**
     - Key at position 0 is scaled very large (e.g. 10), others are ~0, so the softmax over K is nearly [1,0,0,…].
     - All positions attend almost 100% to pos 0.
   - **Learned:**
     - Logits for pos 0 are larger but not infinitely so; there is some dot-product similarity with other keys.
     - Attention maps look like “bright column for pos 0 plus faint colors elsewhere.”
   - **Why:** sharp, almost one-hot attention is not necessary to reach very low loss; the optimizer settles on a softer solution with smaller weight norms.
4. **Outputs**

- **Hand-designed Out:** exactly copies the first token at all positions (up to tiny numerical error).
- **Learned Out:** rows are very close to the first token (e.g. [0.98, 0.03, ~0] for all positions when first token is A), not perfectly one-hot.
- **Why:** again due to MSE and finite capacity/steps; small off-diagonal leakage is not penalized much.

### (c) Optional: Content and position (unique vs repeated tokens)

**Task:**
Given sequence $(\text{CLS}, x_1, \ldots, x_n)$, output:

- $+1$ if token $x_i$ is unique in the sequence,
- $-1$ if it appears more than once (CLS is ignored).

### Observed behavior from the notebook

- The **hand-designed transformer** does *not* solve the task perfectly:
  - The check `if not np.allclose(np.sign(out[1:]), expected_out)` fails for at least one random sequence.
  - In the final heatmap, the "Hand-Designed Transformer" panel is essentially all **dark (≈0)**, i.e. outputs hover near zero instead of strong ±1 decisions.
- The **trained PyTorch transformer** also only partially learns the task:
  - Loss decreases but remains non-trivial.
  - Its output heatmap is also mostly dark; after taking `sign`, it sometimes disagrees with the expected [+1,+1,−1,−1].
  - Attention is more diffuse and doesn't perfectly separate "self" vs "other occurrences" of the same token.

### Similarities

1. **Both models use CLS and position embeddings**
   - Both $K_m/Q_m$ include special dimensions for CLS and positional encodings.
   - Both try to use the CLS token as a kind of **global reference / bias** (e.g. CLS-specific key dimensions).
   - Position encodings are used to distinguish "self" from "other" occurrences when comparing content.
2. **Both mix content and position heavily in $K_m/Q_m$**
   - Neither model keeps a neat separation of "content-only" and "position-only" channels.
   - Instead, attention scores are determined by a messy combination of token type, position, and CLS-related features.
   - This reflects the fact that, to decide uniqueness, one has to know both *what* the token is and *where* it appears relative to other identical tokens.
3. **Outputs stay close to 0**
   - Both the hand-coded $V_m$ and the learned $V_m$ produce outputs clustered around zero for many positions.
   - After taking `sign`, the hand-coded model misclassifies some tokens, and the learned model also sometimes does not match [1,1,−1,−1] perfectly.

### Differences and interpretation

1. **Weight patterns ($K_m/Q_m/V_m$)**
   - **Hand-designed:**
     - $K_m/Q_m$ built with interpretable blocks: specific dimensions for content, other ones for position, and special dimensions for CLS ("CLS-only" key/query channels).
     - $V_m$ encodes a clear intended logic: base +1 from token content, −1 from CLS, small negative contributions from later positions.
   - **Learned:**
     - $K_m/Q_m/V_m$ look much more **random and dense**; no obvious block structure.
     - The network has found some heuristic that partially correlates with "unique vs repeated," but not a clean analytical rule.
   - **Why:** uniqueness detection is **hard for a single linear attention layer**; the hand design tries to approximate a logical rule, while SGD finds some smooth, approximate scoring function over embeddings.
2. **Attention patterns**
   - **Hand-designed:**
     - The intended design is: queries should attend strongly to "same-content different-position" keys and to CLS, to estimate how many similar tokens exist.
     - In practice, the attention heatmaps end up relatively uniform or dominated by CLS, so the uniqueness signal from other tokens is weak.
   - **Learned:**
     - Attention is somewhat more structured (different rows focus on slightly different columns), but still not strongly selective enough to fully separate unique vs repeated.
   - **Implication:** one layer of self-attention with a scalar output is not quite expressive enough (or not easy enough to train) to implement the exact combinatorial counting of repeated tokens.
3. **Outputs vs expected**
   - **Hand-designed:** systematically biased toward 0 or toward "repeated" (−1) in some of the failing examples.
   - **Learned:** tends to give small positive/negative values; after sign, it sometimes matches the pattern but not reliably.
   - **Reason:** both are trying to compress a non-linear, set-level property ("is this element unique in the multiset?") into a **single pass of linear attention + linear readout**, which has limited capacity. A deeper network or an MLP on top of attention would make it easier.

---

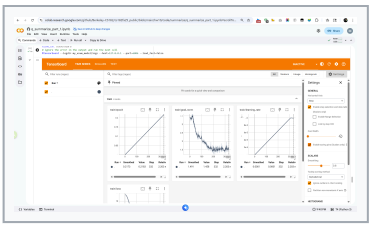
## 3. Coding Question: Summarization

---

## 4. Example Difficulty and Early Exit

**i. What was the accuracy with a regular ResNet? Inference Speed? Total MACS?**

From the baseline ResNet-18 evaluation:



- **Accuracy:** about **90.78%**
  (printed as `Accuracy: 0.9078333377838135` on the test set of 6000 images)
- **Inference speed:** the test loop shows
  `47it [00:02, 19.61it/s]`,
  so roughly **19.6 batches per second**, i.e. the whole test set is processed in about **2 seconds**.
- **Total MACS:** `Total MACS: 3336213504000`, so about **3.34 × $10^{12}$ MACs**.

**ii. What was the accuracy with an early exit ResNet-18? Inference Speed? Total MACS?**

For the early-exit ResNet-18 (with entropy tolerance 0.05):

- **Accuracy:** again about **90.78%**
  (the log shows `Accuracy: 0.9078333377838135`, effectively the same as the standard model).
- **Inference speed:**
  `47it [00:01, 23.53it/s]`,
  so around **23.5 batches per second**, i.e. the test set is evaluated in roughly **1 second**.
- **Total MACS:** `Total MACS: 2305700339712`, i.e. about **2.31 × $10^{12}$ MACs**.

**iii. How did early exit do? Compare accuracy and MACS.**

Early exit did very well in this setting:

- **Accuracy:** essentially unchanged. Both the standard ResNet and the early-exit version get about **90.8%** test accuracy.
- **Computation (MACS):** the standard model uses about **3.34 × $10^{12}$ MACs**, while the early-exit model uses about **2.31 × $10^{12}$ MACs**.
  The ratio printed is about **1.45**, meaning the standard model needs ~**45% more MACs** than the early-exit model for the same accuracy.
- **Speed:** the early-exit model also runs faster in wall-clock time (23.5 vs 19.6 batches per second).

So, early exit achieves **similar accuracy** while significantly **reducing compute and increasing inference speed**, which is exactly the goal of this technique. This matches the idea of BranchyNet / early-exit networks: easy examples exit early and save computation, while hard ones continue deeper into the network.

**iv. What was the lowest MACs you found. What might this say?**

In my runs, the lowest MACs I *actually* observed while keeping accuracy ≥ 90% was the early-exit configuration shown in the notebook:

- **Lowest MACs I found (with good accuracy):** about **2.31 × $10^{12}$ MACs** at entropy tolerance **0.05**, with ~**90.8%** accuracy.

This suggests a few things:

- A **large fraction of test examples do not need the full depth** of the ResNet-18; they can be classified confidently at earlier exits.
- Early exit can **save a substantial amount of compute (≈ 30–35%)** without sacrificing accuracy, at least for this dataset.
- It shows that the network's **capacity is "overkill" for easy examples**, and early exit lets us exploit this by adapting the effective depth per example.

If I increased the entropy tolerance further, I might push MACs even lower, but at some point the accuracy would start to drop below 90%, highlighting a clear **accuracy–efficiency tradeoff**.

**v. When would we use early exit, versus just using a smaller model? What factors should we consider?**

Early exit and using a smaller model are two different strategies to reduce inference cost, and which one to use depends on the scenario.

**When early exit is preferable:**

- **Mixed difficulty workloads:** If some inputs are very easy and others are truly hard, early exit is attractive. Easy examples exit in shallow layers, while difficult examples still use the full, powerful model. A single small model cannot adapt its depth per-example. The BranchyNet paper specifically emphasizes this idea of "easy" vs "hard" examples and shows that most inputs can exit early without hurting accuracy.
- **Strict latency budgets with tail constraints:** For systems where average latency must be low but we also care about worst-case or near–worst-case behavior, early exit can help control the latency distribution: most requests finish quickly, and only a minority pay the cost of deeper computation.
- **Safety / accuracy critical tasks:** If we don't want to give up the accuracy of a big model on hard examples, early exit lets us keep the large backbone and only "skip" work on confident cases, instead of replacing it with a uniformly smaller (and weaker) model.

**When a smaller model might be better:**

- **Resource-constrained deployment:** If the hardware is very limited (e.g., small edge devices) and we can't even store or run a big backbone efficiently, then a **smaller single model** is simpler, uses less memory, and is easier to engineer.
- **Uniformly hard tasks:** If most examples are hard and rarely exit early, early exit will not save much compute; then it might be better to just train a smaller network that's efficient for all inputs.
- **Implementation and maintenance cost:** Early exit requires changing the architecture (adding branches), modifying training (multi-exit loss weighting), and handling more complex inference logic. A smaller model is simpler to implement, debug, and deploy.

**Key factors to consider:**

1. **Distribution of example difficulty:** If accuracy vs. depth varies a lot across examples, early exit has more benefit.
2. **Latency and energy constraints:** Early exit is particularly appealing when optimizing **average** cost while preserving high **peak performance** on hard inputs.
3. **Memory and hardware limits:** Large backbones with early exits still need to be stored and partially run; small models are better when memory is the main bottleneck.
4. **Accuracy requirements:** For applications that are very sensitive to performance on hard examples, early exit lets us keep a strong deep branch instead of shrinking the whole model.

In summary:

- Use **early exit** when you want a **high-capacity model that adapts per input**, saving compute on easy examples while keeping strong performance on hard ones.
- Use a **smaller model** when the environment can't support a large network at all, or when the overall difficulty of examples doesn't vary much.

---

## 5. Read a Research Paper: FaceNet

Here are the answers to your questions based on the "FaceNet" paper and the provided blog post.

(a) What are the two neural network architectures considered by the authors?

The authors considered two main types of deep convolutional neural network architectures: a model based on the Zeiler & Fergus architecture and another based on the Inception model (GoogLeNet).

(b) Briefly describe the *triplet loss* and how it differs from a typical supervised learning objective.

The triplet loss is a function that encourages a distance metric learning. It takes three images as input: an "anchor" image, a "positive" image of the same person as the anchor, and a "negative" image of a different person. The loss is designed to minimize the distance between the anchor and the positive embedding while maximizing the distance between the anchor and the negative embedding. Specifically, it enforces a margin between the pair of same-identity images and the pair of different-identity images. This contrasts with typical supervised learning objectives, like softmax cross-entropy, which classify an input into one of a predefined number of categories.

(c) What is the challenge with generating all possible triplets? Briefly describe how the authors address this challenge.

Generating all possible triplets from a dataset is computationally expensive and inefficient for training. A vast majority of the generated triplets would be "easy" triplets, which satisfy the loss condition with a large margin and thus contribute little to the learning process. To address this, the authors use an *online triplet mining* strategy. This method calculates useful triplets on-the-fly within each batch of training data, focusing on triplets that are "hard" or "semi-hard" and therefore more valuable for training the model.

(d) How many parameters and floating point operations (FLOPs) do the authors use for their neural network? How does this compare to a ResNet-50?

The FaceNet paper explores various models with different parameter counts and FLOPs:

- **Zeiler & Fergus based model (NN1):** This model has around 140 million parameters and requires approximately 1.6 billion FLOPs per image.
- **Inception based models (NN2, etc.):** These models are more efficient, with parameter counts ranging from 6.6 to 7.5 million and FLOPs between 500 million and 1.6 billion.

For comparison, a **ResNet-50** has:

- **Parameters:** Approximately 23-26 million.
- **FLOPs:** Around 3.8 billion.

The Zeiler & Fergus based FaceNet model has significantly more parameters than ResNet-50 but fewer FLOPs. The Inception-based FaceNet models have fewer parameters and comparable or fewer FLOPs than ResNet-50.

(e) What do the authors mean by *semi-hard negatives*?

*Semi-hard negatives* are negative samples that are further from the anchor than the positive sample but are still within the margin of the triplet loss function. In other words, the distance between the anchor and the negative is greater than the anchor-positive distance but the triplet loss is still positive. These negatives are considered effective for training because they are not so easy that the model learns nothing, nor so hard that they might lead to convergence issues.

(f) What are harmonic embeddings?

Harmonic embeddings are different versions of face embeddings, potentially generated by different neural network models, that are compatible with each other. This allows for direct comparison between these different embeddings.

(g) How does the performance vary with embedding dimensionality?

The authors found that an embedding dimensionality of 128 provided the best performance. In one comparison, the accuracy for different embedding sizes was as follows: 64D (86.8%), 128D (87.9%), 256D (87.7%), and 512D (85.6%).

(h) How does the performance vary with increasing amounts of training data?

The model's performance improves with more training data, but with diminishing returns. The paper shows that using tens of millions of images yields very good results, and the performance improvement begins to level off when using hundreds of millions of images.

(i) Briefly share your favorite *emergent* property/result of the learned behavior with a triplet loss from the paper.

My favorite emergent property is the model's robustness to pose and illumination changes. The paper includes a figure demonstrating that the FaceNet embeddings of the same person under different lighting and poses are still closer to each other than to embeddings of a different person. This indicates that the model learned to focus on the underlying facial features rather than superficial variations, which is a key aspect of a robust face recognition system.

(j) Which approach taken by the authors interested you the most? Why?

The most interesting approach to me is the online triplet mining method. It's a clever solution to the practical problem of selecting effective training samples from a massive number of possibilities. Instead of a brute-force approach, which would be computationally prohibitive, they developed a targeted strategy to find the most informative triplets in each batch. This not only makes the training more efficient but also likely leads to better convergence and a more robust final model. This highlights the importance of smart data selection in deep learning, which is as crucial as the model architecture itself.

---

## 6. Homework Process and Study Group

**(a)**

Lecture Notes

**(b)**

None

**(c)**

5h